

Order Management System

DBMS Task – Week 4

1. Introduction

An Order Management System is a database application used to manage customer orders and the products included in each order. It stores order dates, product quantities, prices and total amounts using Orders and Order_Details tables.

2. Objectives

- Design Orders and Order_Details tables.
- Manage customer product orders.
- Store order date, quantity and total amount.
- Perform insertion and modification operations.
- Generate customer order history reports.
- Maintain data integrity using keys.

3. Orders Table

```
Order_ID INT PRIMARY KEY
Customer_ID INT NOT NULL
Order_Date DATE NOT NULL
Total_Amount DECIMAL(10,2) NOT NULL
```

4. Order_Details Table

```
Order_Detail_ID INT PRIMARY KEY
Order_ID INT NOT NULL
Product_ID INT NOT NULL
Quantity INT NOT NULL
Unit_Price DECIMAL(10,2) NOT NULL
Subtotal DECIMAL(10,2) NOT NULL
```

5. Table Creation

```
CREATE TABLE Orders (
  Order_ID INT PRIMARY KEY,
  Customer_ID INT NOT NULL,
  Order_Date DATE NOT NULL,
  Total_Amount DECIMAL(10,2) NOT NULL
);

CREATE TABLE Order_Details (
  Order_Detail_ID INT PRIMARY KEY,
  Order_ID INT NOT NULL,
  Product_ID INT NOT NULL,
  Quantity INT NOT NULL,
  Unit_Price DECIMAL(10,2) NOT NULL,
  Subtotal DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
);
```

6. Order Insertion

```
INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount) VALUES (101, 1,
'2026-08-17', 1500.00);
```

```
INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal) VALUES (1, 101, 10, 2, 500.00, 1000.00);
```

```
INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal) VALUES (2, 101, 15, 1, 500.00, 500.00);
```

7. Order Modification

```
UPDATE Order_Details SET Quantity = 3, Subtotal = 1500.00 WHERE Order_Detail_ID = 1;
```

```
UPDATE Orders SET Total_Amount = 2000.00 WHERE Order_ID = 101;
```

8. Customer Order History Report

```
SELECT o.Order_ID, o.Order_Date, od.Product_ID, od.Quantity, od.Unit_Price, od.Subtotal, o.Total_Amount FROM Orders o JOIN Order_Details od ON o.Order_ID = od.Order_ID WHERE o.Customer_ID = 1 ORDER BY o.Order_Date DESC;
```

9. Calculate Total Amount

```
SELECT Order_ID, SUM(Subtotal) AS Total_Amount FROM Order_Details GROUP BY Order_ID;
```

10. Relationship

Customers → Orders → Order_Details → Products. One customer can place many orders, and one order can contain many order-detail records.

11. Advantages

- Efficient order management.
- Reduced data duplication.
- Data integrity through primary and foreign keys.
- Easy order modification.
- Customer purchase-history reporting.
- Automatic total calculation support.

12. Conclusion

The Order Management System efficiently stores and manages customer orders. Orders stores general order information, while Order_Details stores products, quantities, prices and subtotals. INSERT, UPDATE and SELECT operations support order management and customer history reports.